API TRACK

Full-stack APIs, *end to end.*

HTTP, browser requests, Express servers, middleware, CORS, and CRUD as the pattern behind most web apps.

HTTP | Express | fetch | axios | CRUD

THE RELATIONSHIP

The client asks. The server answers.

The browser initiates an HTTP request. The server handles it and sends exactly one HTTP response.

THE CONTRACT

Requests and responses have predictable parts.

REQUEST

- ✓ Method: GET, POST, PATCH
- ✓ Path: /api/posts/7
- ✓ Headers: metadata
- ✓ Body: data for create/update

RESPONSE

- ✓ Status code: what happened
- ✓ Headers: metadata
- ✓ Body: data or error details
- ✓ Exactly one response per request

HTTP is the grammar of frontend-backend conversation.

METHODS

Method plus path communicates intent.

Request	Meaning	CRUD
GET /api/posts	Give me all posts	Read many
GET /api/posts/2	Give me post 2	Read one
POST /api/posts	Here is a new post	Create
PATCH /api/posts/7	Update part of post 7	Update
DELETE /api/posts/4	Remove post 4	Delete

STATUS CODES

Status codes tell the frontend what happened.

200 OK
Successful read or update.

201 CREATED
New resource was created.

204 NO CONTENT
Delete succeeded, no body.

400 BAD REQUEST
Client sent invalid data.

404 NOT FOUND
Resource does not exist.

500 SERVER ERROR
Server crashed or failed.

FETCH

fetch() starts an HTTP request.

```
async function fetchPosts() {
  const response = await fetch(
    "https://jsonplaceholder.typicode.com/posts"
  )
  const data = await response.json()
  console.log(data)
}
```

PROMISE REMINDER

- ✓ Pending: still working.
- ✓ Fulfilled: succeeded with a value.
- ✓ Rejected: failed with an error.
- ✓ `await` pauses inside an async function.

REACT GOTCHA

Do not fetch in the component body.

INFINITE LOOP

- × Component renders.
- × Fetch runs immediately.
- × `setState` runs.
- × Component renders again.
- × Fetch runs again.

USE EFFECT

- ✓ Render first.
- ✓ Run side effect after mount.
- ✓ Set data into state.
- ✓ Render with the loaded data.

render should describe UI, not kick off endless work.

USEEFFECT

Fetch after the component mounts.

```
import { useEffect, useState } from "react"

const App = () => {
  const [posts, setPosts] = useState([])

  useEffect(() => {
    async function loadPosts() {
      const res = await fetch("/api/posts")
      setPosts(await res.json())
    }
    loadPosts()
  }, []) // run once after mount
}
```

AXIOS

Axios makes common requests less noisy.

```
import axios from "axios"

const res = await axios.get("/api/posts")
setPosts(res.data)

await axios.post("/api/posts", {
  title,
  body
})
```

WHY USE IT?

- ✓ JSON is already parsed at `res.data`.
- ✓ Request bodies are straightforward.
- ✓ Errors are easier to catch consistently.
- ✓ Still returns a Promise.

SERVER SIDE

Express lets Node speak HTTP.

Express is an NPM library for building servers. It listens for requests and runs route handlers.

```
const express = require("express")
const app = express()

app.get("/", (req, res) => {
  res.send("Hello World")
})

app.listen(8080)
```

ROUTING

Express matches method and path.

```
app.get("/api/posts", (req, res) => { ... })
app.get("/api/posts/:id", (req, res) => { ... })
app.post("/api/posts", (req, res) => { ... })
app.patch("/api/posts/:id", (req, res) => { ... })
app.delete("/api/posts/:id", (req, res) => { ... })
```

route order matters when paths overlap.

RESPONSES

Every route sends exactly one response.

```
res.send("Hello")
res.json({ id: 1, title: "Intro" })
res.sendStatus(201)
res.status(400).json({ error: "Bad input" })
```

COMMON BUGS

- × Returning data instead of sending it.
- × Sending twice in the same code path.
- × Forgetting to `return` after a 404.
- × Never sending a response, so the client hangs.

READING REQUESTS

Data enters routes in a few places.

REQ.BODY

JSON sent with POST, PATCH, or PUT. Requires `express.json()`.

REQ.PARAMS

Named path segments like `/posts/:id`. Values are strings.

REQ.QUERY

URL search params like `?page=2`.

```
const id = Number(req.params.id)  // build the habit
```

BETWEEN REQUEST AND ROUTE

Middleware prepares the request.

```
app.use(express.json())  // parse JSON bodies
app.use(cors({ origin: "http://localhost:3000" }))
app.use(morgan("dev"))
app.use("/api", apiRouter)
```

MENTAL MODEL

- ✓ Request comes in.
- ✓ Middleware runs in order.
- ✓ Each middleware can modify `req`.
- ✓ `next()` passes control forward.

BROWSER SECURITY

Different origin? The server must allow it.

ORIGIN

Protocol + hostname + port. `localhost:3000` and `localhost:8080` are different origins.

```
const cors = require("cors")

app.use(cors({
  origin: "http://localhost:3000"
}))
```

CORS is not an axios problem. It is a browser-enforced server permission.

THE APPLICATION PATTERN

CRUD is most apps in four verbs.

CREATE

POST

Insert a new record.

READ

GET

Load one or many records.

UPDATE

PATCH

Change part of a record.

DELETE

DELETE

Remove a record.

if you can build CRUD, you can build a surprising amount of the web.

BACKEND ROUTER

One resource gets one router.

```
const router = require("express").Router()
const { Post } = require("../models")

router.get("/", async (req, res) => {
  const posts = await Post.findAll({ order: [["createdAt", "DESC"]] })
  res.json(posts)
})

router.post("/", async (req, res) => {
  const post = await Post.create(req.body)
  res.status(201).json(post)
})

router.patch("/:id", async (req, res) => {
  const post = await Post.findByPk(Number(req.params.id))
  if (!post) return res.sendStatus(404)
  res.json(await post.update(req.body))
})

router.delete("/:id", async (req, res) => {
  const post = await Post.findByPk(Number(req.params.id))
  if (!post) return res.sendStatus(404)
  await post.destroy()
  res.sendStatus(204)
})
```

FRONTEND + DEBUGGING

Update UI state to match server truth.

```
const fetchPosts = async () => {
  const res = await axios.get("/api/posts")
  setPosts(res.data)
}

const createPost = async () => {
  const res = await axios.post("/api/posts", { title, body })
  setPosts([...posts, res.data])
}

const updatePost = async (id, updates) => {
  const res = await axios.patch(`/api/posts/${id}`, updates)
  setPosts(posts.map((p) => (p.id === id ? res.data : p)))
}

const deletePost = async (id) => {
  await axios.delete(`/api/posts/${id}`)
  setPosts(posts.filter((p) => p.id !== id))
}
```

DEBUG IN LAYERS

- ✓ Postman first: does the API route work?
- ✓ Server terminal: did Express receive the request?
- ✓ Network tab: what status and body came back?
- ✓ React state: did `setState` receive the shape you expected?
- ✓ Common fixes: await promises, check null, return after 404, send one response.